

Junta de Andalucía
Consejería de Educación y Deporte

UD 3
Funciones y Procedimientos
–
Módulo de Programación
1º DAW

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Autor: Fran Gómez

Objetivos

Al finalizar la unidad, serás capaz de:

- Comprender la modularidad y sus ventajas en el desarrollo de software.
- Descomponer un problema en subproblemas y resolverlos mediante métodos.
- Definir métodos en Java utilizando correctamente su tipo de retorno, nombre, parámetros y cuerpo.
- Diferenciar entre parámetros formales y argumentos.
- Utilizar parámetros para comunicar información a los métodos.
- Comprender cómo se pasan los argumentos en Java y distinguir entre la modificación de variables primitivas y la modificación del estado de objetos.
- Devolver resultados mediante `return` y distinguir los métodos que devuelven valores de los que utilizan `void`.
- Definir y utilizar métodos estáticos mediante `static`.
- Reconocer cuándo resulta adecuado utilizar un método estático.
- Diseñar programas sencillos con una estructura modular, legible y reutilizable.
- Explicar y justificar oralmente las decisiones adoptadas al desarrollar un programa.



Contenidos y CE

Bloque	Contenidos	Criterios de evaluación	Evidencias de aprendizaje
B1. Modularidad	Descomposición de problemas, responsabilidades, reutilización y legibilidad	RA4.c	Diseña un programa organizado en métodos con responsabilidades claras.
B2. Definición de métodos	Sintaxis, nombre, parámetros, cuerpo y tipo de retorno	RA4.c	Define métodos correctamente y los integra en una clase.
B3. Parámetros	Parámetros formales, argumentos, paso por valor y referencias a objetos	RA2.f	Invoca métodos pasando los argumentos adecuados y explica su comportamiento.
B4. Retorno de valores	`return`, tipos de retorno, `void` y utilización de resultados	RA4.c	Implementa métodos que calculan y devuelven resultados.
B5. Métodos estáticos	`static`, llamadas mediante la clase y métodos de utilidad	RA2.e, RA4.h	Define e invoca métodos estáticos correctamente.
B6. Integración	Combinación de métodos, pruebas y refactorización	RA2.e, RA2.f, RA4.c, RA4.h	Resuelve un problema modular y justifica oralmente su solución.

Bloque 1.

Modularidad: dividir para resolver

Objetivo: comprender por qué necesitamos métodos y cómo ayudan a organizar un programa.

Bloque 1. Modularidad: dividir para resolver

```java
public class Tienda {
    public static void main(String[] args) {
        double precio1 = 100;
        double descuento1 = 10;
        double total1 = precio1 - precio1 * descuento1 / 100;

        double precio2 = 250;
        double descuento2 = 15;
        double total2 = precio2 - precio2 * descuento2 / 100;

        System.out.println("Total 1: " + total1);
        System.out.println("Total 2: " + total2);
    }
}
```

¿Qué ocurre cuando un programa crece?

Bloque 1. Modularidad: dividir para resolver


```java
public class Tienda {

    public static double calcularPrecioFinal(
            double precio, double descuento) {

        return precio - precio * descuento / 100;
    }

    public static void main(String[] args) {
        double total1 = calcularPrecioFinal(100, 10);
        double total2 = calcularPrecioFinal(250, 15);

        System.out.println("Total 1: " + total1);
        System.out.println("Total 2: " + total2);
    }
}
```

La solución: **dividir** el problema

Ideas clave:

- Un método **encapsula** una tarea concreta.
- Podemos **reutilizarlo** tantas veces como necesitemos.
- Cada método debe tener **una responsabilidad clara.**
- La modularidad facilita la lectura, las pruebas y el **mantenimiento** del código.

Ejercicio 1: Detectives del código (CE: 4c)

A continuación verás un programa con con código duplicado e ineficiente:

1. Localiza las instrucciones repetidas: Identifica qué bloques de código realizan exactamente la misma tarea con distintas variables.

2. Describe con tus palabras qué hacen esas instrucciones repetidas y por qué crees que no es eficiente escribir el programa de esta manera.

3. Piensa un método que calcule el área de un rectángulo, ¿qué recibiría? ¿qué devolvería?

4. Refactorizar el programa completo para eliminar duplicaciones y mejorar la organización. Supón que el método es:

```
public static int calcularAreaRectangulo(int base, int altura) { return base * altura; }
```

Bloque 2.

Definición y estructura de los métodos

Objetivo: aprender a declarar métodos en Java y comprender cada uno de sus componentes.

Sintaxis de un método

```
public static double calcularMedia(int nota1, int nota2) {
    return (nota1 + nota2) / 2.0;
}
```

Modificador de acceso	`public`	Determina desde dónde puede accederse al método.
Modificador de clase	`static`	Permite invocarlo sin crear un objeto de esa clase.
Tipo de retorno	`double`	Indica el tipo de resultado que devuelve.
Nombre	`calcularMedia`	Identifica el método.
Parámetros	`int nota1, int nota2`	Declaran los datos que recibe.
Cuerpo	Entre `{ }`	Contiene las instrucciones.
Retorno	`return ...;`	Devuelve el resultado al punto de llamada.

Convenciones para nombrar métodos

En Java se utiliza habitualmente *lower camel case*:

- `calcularPrecio()`
- `mostrarResultado()`
- `esMayorDeEdad()`
- `obtenerNotaMedia()`

Los nombres deben expresar acciones o resultados concretos.

Evitar nombres ambiguos como `hacer()`, `proceso()` o `dato()` cuando no describen claramente la finalidad del método.

Bloque 2. Definición y estructura de los métodos

Métodos que realizan tareas y métodos que calculan resultados

Método que realiza una acción:

```java
public static void mostrarSaludo() {
    System.out.println("Bienvenido a DAW");
}
```

Método que calcula y devuelve un resultado:

```java
public static int sumar(int a, int b) {
    return a + b;
}
```

Ejercicio 2:

Dadas las dos declaraciones incompletas, escríbelas rellenando lo que falta

```
public static ______ calcularDoble(int numero) {
    return numero * 2;
}
```

```
public static ______ mostrarMensaje() {
    System.out.println("Hola");
}
```

Bloque 3. Parámetros y argumentos

Objetivo: comprender cómo se transmite información a los métodos.

¿Para qué sirven los parámetros?

```
public static void saludar(String nombre) {
    System.out.println("Hola, " + nombre);
}

saludar("Ana");
saludar("Carlos");
saludar("María");
```

El método se reutiliza con diferentes datos.

Es importante distinguir:

- **Parámetro formal:** variable declarada en la definición del método; en el ejemplo, `nombre`.
- **Argumento:** valor o expresión proporcionado en la llamada; por ejemplo, `"Ana"`.

Bloque 3. Parámetros y argumentos

Métodos con varios parámetros

```java
public static double calcularDescuento(
        double precio, double porcentaje) {

    return precio * porcentaje / 100;
}
```

```java
double descuento = calcularDescuento(80, 15);

System.out.println(descuento);
```

Los argumentos se asocian con los parámetros según su **posición** y que deben ser compatibles con los **tipos** declarados.

Bloque 3. Parámetros y argumentos

¿Qué pasa cuando los argumentos no son correctos?

```
calcularDescuento(80);
```

```
calcularDescuento(80, 15, 5);
```

No encontrará una declaración que se corresponda.

Bloque 3. Parámetros y argumentos

Ejercicio 3 (CE: 2f)

Diseñar un método que calcule el importe de una compra aplicando un descuento.

Nivel	Requisitos
Iniciación	Recibir un precio y devolver el precio con un descuento fijo.
Intermedio	Recibir el precio y el porcentaje de descuento.
Ampliación	Recibir el precio, el descuento y un porcentaje adicional de impuesto, y calcular el importe final.

¿Puede un método modificar una variable?

```
public class Ejemplo {

    public static void modificar(int numero) {
        numero = 100;
    }

    public static void main(String[] args) {
        int valor = 10;

        modificar(valor);

        System.out.println(valor);
    }
}
```

Bloque 5. Retorno de valores

Objetivo: utilizar métodos para calcular resultados que puedan emplearse en otras operaciones.

El método devuelve un resultado

```java
public static int sumar(int a, int b) {
    return a + b;
}
```

Podemos utilizar el resultado de diferentes maneras:

```java
int resultado = sumar(5, 3);

System.out.println(resultado);

System.out.println(sumar(10, 20));

int total = sumar(sumar(2, 3), 4);
```

Bloque 5. Retorno de valores

Devolver no es lo mismo que mostrar

```java
public static void mostrarSuma(int a, int b) {
    System.out.println(a + b);
}
```

Procedimiento

```java
public static int calcularSuma(int a, int b) {
    return a + b;
}
```

Función

El primer método muestra el resultado. El segundo lo devuelve para que el programa que lo llama decida qué hacer con él.

Por ejemplo, si necesitamos reutilizar la suma:

```java
int resultado = calcularSuma(4, 6);
int resultadoDoble = resultado * 2;
```

Ejercicio 4: Calculadora modular (ce 4c)

Crear una clase con métodos para:

- Sumar.
- Restar.
- Multiplicar.
- Dividir.

Para la división, se debe decidir cómo tratar el divisor cero.